IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 19 February 2026, accepted 26 February 2026, date of publication 2 March 2026, date of current version 11 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3669449

APPLIED RESEARCH

YOLO-AUG: Enhancing Wildlife Monitoring on Far-Edge Devices With Dynamic Color-Space Augmentation and Efficient Learning

JEAN PIERRE ASDIKIAN[1], (Graduate Student Member, IEEE), SEBASTIAN TROIA[1], (Member, IEEE), OMRAN AYOUB[2], (Member, IEEE), KIVILCIM YILMAZ[3], GIACOMO SGUOTTI[1], (Graduate Student Member, IEEE), MENGYAO LI[1], (Graduate Student Member, IEEE), MARCO MARCON[1], (Member, IEEE), AND GUIDO MAIER[1], (Senior Member, IEEE)

[1]Department of Electronics, Information and Bioengineering, Politecnico di Milano, 20133 Milan, Italy
[2]University of Applied Sciences and Arts of Southern Switzerland, 6962 Lugano, Switzerland
[3]Ankara University, 06100 Ankara, Türkiye

Corresponding author: Jean Pierre Asdikian (jeanpierre.asdiguian@polimi.it)

This work was supported by European Union—Next Generation EU through Italian National Recovery and Resilience Plan (NRRP), Mission 4, Component 2, Investment 1.3, partnership on "Telecommunications of the Future" (PE00000001—program "Research and Innovation on Future Telecommunications Systems and Networks, to make Italy more Smart (RESTART)") under Grant CUP D43C22003080001.

ABSTRACT Edge and far-edge computing devices are increasingly employed in applications requiring localized, low-latency processing despite their limited computational resources. One such application is wild animal monitoring in agricultural environments, which is vital for crop protection and disease prevention. Deep Learning (DL)-based models, especially those designed for image recognition, provide highly effective tools for identifying and monitoring wildlife. However, their performance is significantly constrained by two key challenges: the limited computational resources of edge and far-edge devices, and the variability in image quality caused by environmental factors. These limitations can compromise the robustness and reliability of the models, reducing their effectiveness in real-world applications. To address these challenges, we empirically evaluate the feasibility and overhead of supervised, transfer-learning, and semi-supervised training regimes for YOLO-based (You Only Look Once) detectors in resource-constrained settings, quantifying the practical trade-offs between detection performance and training overhead. Within this evaluation, we define and assess YOLO-AUG, a lightweight training-time augmentation module integrated into the YOLO training pipeline that applies on-the-fly multi-color-space transformations (e.g., Hue/Saturation/Value (HSV) and Commission Internationale de l'Éclairage Lab (CIELab) variants) to improve robustness in wildlife monitoring, without modifying the detector architecture. We also analyze memory constraints and power consumption on representative far-edge hardware (e.g., Graphical Processing Unit (GPU)-equipped trap cameras) and provide practical deployment guidelines that quantify trade-offs between accuracy, training time, and resource usage. Our evaluation underscores the trade-offs between these methodologies to provide robust and efficient monitoring solutions for our boar, wolf and deer detection scenario, moving toward a more automated workflow that reduces human labeling effort in our target deployment setting, crucial for both agricultural management and wildlife conservation.

INDEX TERMS Deep learning, far-edge computing, image preprocessing, object detection, semi-supervised learning, supervised learning, transfer learning, wildlife, YOLO.

I. INTRODUCTION

Monitoring wild animals in agricultural settings pose significant challenges, necessitating innovative solutions for effective management and conservation. Traditional methods,

The associate editor coordinating the review of this manuscript and approving it for publication was Ines Domingues.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

such as fences and manual monitoring, often prove inadequate as animals frequently bypass physical barriers and roam freely across properties, causing extensive crop damage, car accidents, and contributing to the spread of diseases such as swine fever [1].

To mitigate these issues, deploying far-edge devices, such as cameras and drones equipped with Deep Learning (DL)-based models for image recognition, can facilitate real-time monitoring and alert systems by connecting to edge and cloud servers. However, these devices face substantial limitations in computational power and resources, which impact their effectiveness. Maintaining accurate and up-to-date DL models for animal detection demands periodic retraining and updates, processes that are both resource-intensive and time-consuming. Furthermore, achieving robust model performance across diverse environmental conditions such as rain, fog, and varying illumination requires access to extensive and highly diverse datasets.

Robustness is not the only challenge during deployment and operation of DL models. Model performance and training efficiency are equally critical, as optimizing these factors can significantly reduce the computational resources required for real-world applications. In other words, fully retraining the model to incorporate updates and redeploy it can be an extremely resource-intensive and time-consuming process, posing significant challenges for practical implementation. Consequently, practical deployments benefit from workflows that quantify the trade-offs between accuracy, training time, and resource usage, especially when occasional retraining is required under constrained hardware.

In this work, we empirically analyze the feasibility and overhead of training and adaptation for YOLO-based (You Only Look Once) detectors in far-edge settings, and quantify the trade-offs between detection performance and training overhead across supervised, transfer-learning, and semi-supervised regimes. Within this evaluation, we implement and assess YOLO-AUG, a training-time augmentation module integrated into the Ultralytics YOLO training pipeline [2], which applies on-the-fly color-space augmentations to encourage robustness to color variation and to emulate sensor or white-balance variability. YOLO-AUG adds seven targeted color-space transformations (Grayscale, Luminance/Chrominance (YUV), XYZ, HSV, Hue/Lightness/Saturation (HLS), Luminance/Red-difference/Blue-difference (YCrCb), and CIELab) in addition to the default Ultralytics augmentations (e.g., HSV jitter and geometric transforms) provided by Ultralytics [2]. Beyond the default HSV/BGR-style color operations typically exposed through standard augmentation controls, YOLO-AUG introduces explicit multi–color-space conversions by extending the training-time preprocessing path. This design is specifically to focus on computationally inexpensive transformations that remain compatible with far-edge retraining, where heavier augmentation pipelines may be impractical under strict memory and runtime constraints. Importantly, our objective is not to propose a new detector architecture, but to isolate the effect of training-time color-space augmentation and quantify its practical cost under constrained hardware.

To address the challenge of model retraining and deployment in resource-constrained environments, we explore the use of transfer learning, where pretrained weights from one model are utilized to accelerate the training of another on a different dataset. While this approach reduces training time, it is not without its limitations, as it often requires human intervention for data collection and labeling.

To attempt to mitigate the latter challenges, we consider semi-supervised learning as an alternative, enabling the automation of the labeling process while allowing the model to continue learning, albeit with some tolerance for noise introduced by mislabeling. This approach reduces the need for human intervention and provides a trade-off between performance and automation, leveraging previously trained weights to approximate the performance of fully supervised models while minimizing human labeling effort.

Finally, given the memory and computational limitations of far-edge devices, such as GPU-equipped trap cameras, we empirically analyze the trade-offs between training time, memory usage, and detection accuracy. Based on this analysis, we provide practical guidelines for configuring YOLO models for real-time wildlife monitoring on constrained hardware.

The main contributions of this work are:

- A dynamic multi-color-space training-time augmentation module (YOLO-AUG) integrated into the Ultralytics YOLO training pipeline, which randomly samples from seven computationally inexpensive color spaces during training without architectural changes.
- A controlled ablation protocol that isolates the effect of YOLO-AUG via paired comparisons between YOLO-AUG (ON) and a YOLO-Baseline (OFF) under identical settings within each training regime, supported by five-fold analysis to assess stability.
- A comparative study of supervised, transfer-learning, and semi-supervised training regimes for YOLOv8n and YOLOv10n on a real wildlife camera-trap dataset, highlighting trade-offs between detection performance, training time, and data annotation requirements.
- An empirical characterization of far-edge training behavior, including training-time overhead, GPU memory behavior, and power consumption during YOLO training on both a laptop RTX 3050 GPU and a Jetson Orin Nano, together with practical deployment-oriented configuration guidelines.

This paper is organized as follows: Section II reviews related work on YOLO, focusing on its application in transfer and semi-supervised learning scenarios and defines the problem statement addressed in this paper. Section III describes the methods employed to tackle the aforementioned challenges. Section IV details the hyperparameters used for the experimental setup and discusses the results. Finally, Section V provides the concluding remarks.

II. BACKGROUND AND RELATED WORKS

The challenge of monitoring wild animals has prompted the development of innovative solutions leveraging DL models, particularly in resource-constrained environments. Among the most widely used real-time object detection architectures is YOLO (You Only Look Once) [3], [4], which has evolved through several iterations to enhance accuracy and speed. YOLO's one-stage detection architecture makes it a preferred choice for real-time applications, including wildlife monitoring, due to its efficiency and scalability.

Recent advancements have explored the integration of YOLO with techniques such as data augmentation, transfer learning and semi-supervised learning to address the limitations of traditional fully supervised approaches.

Data augmentation has been widely explored in various applications to improve the performance of YOLO-based models by expanding or enhancing training datasets. For instance, the authors of [5] combined CycleGAN [6] with the then-standard YOLOv3 detector [7] to generate synthetic images, transforming CT (Computed Tomography) images into MRI (Magnetic Resonance Imaging) images.

In another study, [8] investigated the use of simple data augmentations, such as rotation and horizontal or vertical flips, to verify the placement of integrated circuits on circuit boards. Their approach achieved an impressive average training accuracy of 98%, with an inference time of less than one second, showcasing the utility of basic augmentation techniques for fast and accurate classification.

Moving beyond simple augmentations, authors in [9] applied advanced techniques like mosaic augmentation for defect detection on the inner walls of coal-fired boilers. The study utilized a modified YOLOv5 architecture that emphasized small object detection by introducing modules that detect basic patterns in an image.

Similarly, [10] leveraged data augmentation techniques, such as rotation, flipping, and zooming, to expand their dataset from 700 to 3000 images. These augmentations improved the detection of shrimp diseases, including White Spot Syndrome (WSS) [11]. Their comparative analysis revealed that YOLOv8 outperformed YOLOv5 highlighting the effectiveness of augmentations combined with the latest YOLO architectures.

To improve generalization capabilities, [12] explored the impact of geometric and photometric transformations on YOLOv5 performance in Remote-Sensed Insect-Trap image detection. They systematically assessed how these augmentations affected the model's performance, offering insights into optimizing augmentations for specific applications.

Further architectural enhancements were explored in [13], where the authors worked on a Jiangnan private garden recognition model. Instead of bloating the dataset, they applied augmentation techniques such as Gaussian noise, random rotations, and occlusions to improve data variability. They also introduced significant modifications to YOLOv8, resulting in a remarkable 15.16% increase in precision.

Building upon data augmentation techniques, transfer learning has become a critical technique in enhancing the efficiency of DL models. By leveraging pretrained weights, transfer learning significantly reduces training time and computational requirements, enabling models to adapt quickly to new datasets. However, this approach often relies on manual data collection and labeling, which can be labor-intensive, costly, and disruptive, especially in field applications like wildlife monitoring where physical access may disturb natural habitats.

A notable application of transfer learning is found in [14], where the detection of roadkilled endangered Brazilian animals was investigated using the BRA-Dataset [15]. The authors performed a comparative analysis of multiple YOLO variants, including YOLOv4 Darknet [16], Scaled-YOLOv4 [17], YOLOv5, YOLOX [18], YOLOv7 [19], and YoloR [20]. Augmentation techniques such as image shifting, flipping, and rotation were used to expand the dataset to 8407 images. Transfer learning was then employed to fine-tune the models, transferring pretrained weights for both training and inference. The results highlighted that Scaled-YOLOv4 performed best in minimizing false negatives, while YOLOv5 achieved the highest frames per second (FPS), making it the fastest among the tested models.

Semi-supervised learning has also emerged as a promising alternative to reduce reliance on manual labeling. In [21], the authors applied semi-supervised learning to a railway defect detection system using YOLOv4 and the Turkish State Railways (TCDD) dataset [22]. By combining a small labeled subset with additional pseudo-labeled images generated through semi-supervised methods, they were able to improve the object detection performance, reducing the need for extensive human intervention.

While far-edge deployment is often motivated by resource constraints, recent surveys emphasize that training and adaptation on edge devices introduce distinct constraints in memory, energy, and runtime overhead compared to inference-only deployments [23], [24]. In addition, on-device learning work highlights the practical limitations of the common train-then-deploy assumption, where serverless adaptation can be constrained by computation and memory budgets [25]. These observations motivate deployment-oriented evaluations that quantify both detection accuracy and the practical cost of training-time strategies under constrained hardware.

Despite these advancements, the reviewed works typically evaluate augmentation, transfer learning, and semi-supervised learning in isolation or under differing experimental settings, which makes it difficult to attribute performance changes to a specific component. In addition, while far-edge deployment is often motivated by resource constraints, in the reviewed works training-time accuracy is less often reported together with training-time resource costs (e.g., energy and memory), which are required to assess on-device adaptation feasibility [23], [24]. This work contributes an evaluation that addresses these gaps by integrating

training-time multi–color-space augmentation, transfer learning, and semi-supervised learning into a unified experimental pipeline for far-edge deployment analysis.

Based on the reviewed literature, we identify three practical gaps that are particularly relevant for far-edge wildlife monitoring with YOLO-based detectors. First, real deployments are affected by substantial appearance variation (e.g., illumination, weather, and sensor/white-balance effects), which can induce domain shift and degrade detection performance. Second, although data augmentation is widely used, a substantial portion of prior work emphasizes offline dataset expansion (e.g., synthetic data generation) or architectural modifications, and the specific contribution of a training-time augmentation module is not always isolated via controlled with/without comparisons under identical training settings. Third, practical far-edge adaptation requires selecting training strategies that balance detection performance against training overhead, memory usage, and power consumption [23], [24].

Accordingly, our objectives are to:

- Integrate a training-time multi-color-space augmentation module (YOLO-AUG) into the Ultralytics YOLO training pipeline without modifying detector architectures.
- Quantify the effect of YOLO-AUG via controlled ablation comparisons under supervised, transfer-learning, and semi-supervised regimes.
- Report performance-resource trade-offs (e.g., training time, memory, and power) on representative far-edge hardware.

III. METHODOLOGY

This study extends our prior work [26] by replacing static, offline augmentation with on-the-fly multi–color-space augmentation integrated into the training loop of YOLOv8 and YOLOv10 [27]. We evaluate YOLO-AUG using F1-score, mAP50, and mAP50-95 [28] under supervised, transfer-learning, and semi-supervised settings, and report associated resource measurements (e.g., memory and energy) on representative far-edge hardware. Figure 1 summarizes the complete experimental workflow used in this study. We use a fixed main split (Train/Val/Test = 60/10/30) for all final reported results, while a separate auxiliary 5-fold cross-validation (80/20 per fold) is performed only on the Train/Val pool to verify the stability of YOLO-AUG gains under repeated resampling. Figure 2 then zooms into the preprocessing stage and illustrates where YOLO-AUG is inserted into the Ultralytics training-time augmentation path. Unless stated otherwise, YOLO-AUG is applied only to training images (Val/Test remain unaugmented). For each learning regime (supervised, transfer-learning, semi-supervised), we report both YOLO-AUG (ON) and YOLO-Baseline (OFF) results under identical settings, where the baseline disables only the proposed YOLO-AUG module while keeping the remaining Ultralytics augmentations unchanged. The latter includes the built-in Albumentations-based photometric transforms provided by Ultralytics which run alongside the standard YOLO augmentations.

To address the computational constraints in edge and far-edge environments and the inevitable model robustness issues, we propose a dynamic data preprocessing function integrated into the Ultralytics training-time augmentation path for detection. This function heavily augments the images without bloating the dataset, enhances color-invariant feature learning and emulates sensor or white-balance variability. This augmentation is enabled in the YOLO-AUG (ON) condition and disabled in the YOLO-Baseline (OFF) condition. All other Ultralytics augmentations, such as Mosaic, HSV jitter, etc., remain unchanged. We use a fixed Train/Val/Test split for all final results, and perform an auxiliary stratified 5-fold CV only on the Train/Val pool to assess robustness under resampling.

We apply this modification to YOLOv8n [2] and YOLOv10n [27] models with pretrained weights, selecting Nano versions for their low resource consumption. YOLOv10 is included for comparison due to its end-to-end Non-Max-Suppression (NMS)-free design and efficiency-driven architecture, which aim to reduce post-processing overhead and inference latency. Models are trained under supervised, transfer, and semi-supervised learning scenarios, each fine-tuned for optimal performance. Training time and computational resources are reported to quantify the trade-offs associated with each learning regime and model choice.

A. YOLO-AUG

YOLO-AUG augments the Ultralytics YOLO detection training pipeline by inserting a set of seven on-the-fly color-space transformations (Grayscale, YUV, XYZ, HSV, HLS, YCrCb, CIELab) into the training-time preprocessing path [26]. These transformations are implemented via OpenCV and are applied in addition to the default Ultralytics detection augmentations (e.g., HSV jitter and geometric transforms). For each epoch, and for each training image, one color-space transform is selected uniformly at random when YOLO-AUG is applied, so that the same image can appear in different color-space representations across epochs. Figure 2 illustrates this process. All transforms preserve the original image resolution and produce three-channel images, so no changes to the model architecture are required.

The reason for which we chose the aforementioned augmentations is as follows. Starting with linear transforms, grayscale is used to isolate luminance and reduce reliance on chroma. YUV/YCrCb are used to split luminance from chroma, thus encouraging robustness to white-balance and color-cast drift. XYZ has also been utilized because it encourages the model to rely less on absolute color and more on structure and contrast. For non-linear spaces, HSV/HLS reparameterize Red-Green-Blue(RGB) so that hue and saturation are decoupled from intensity, offering a complementary view of chroma versus luminance. CIELab

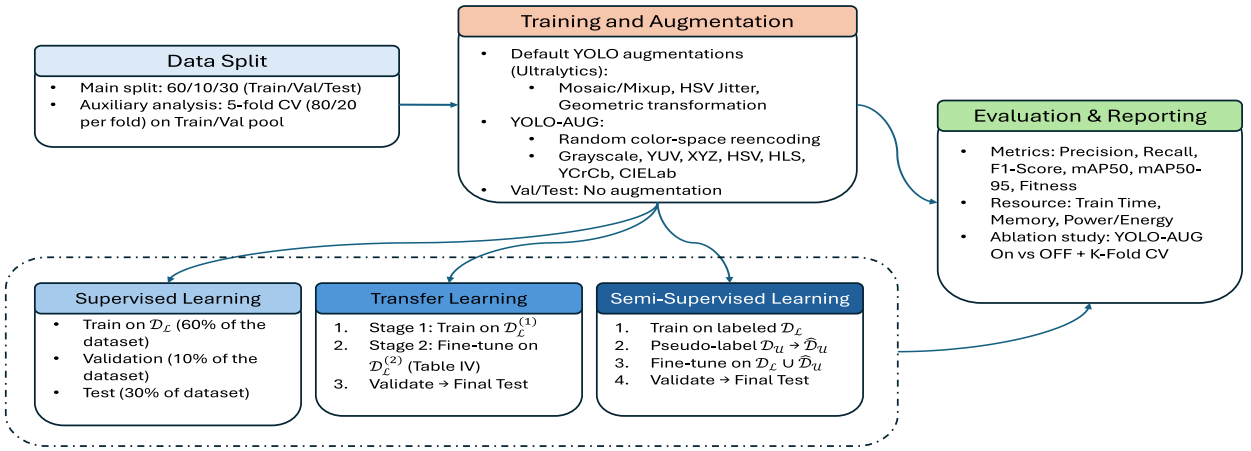


FIGURE 1. Overall evaluation workflow for YOLO-AUG (data splits and training regimes).

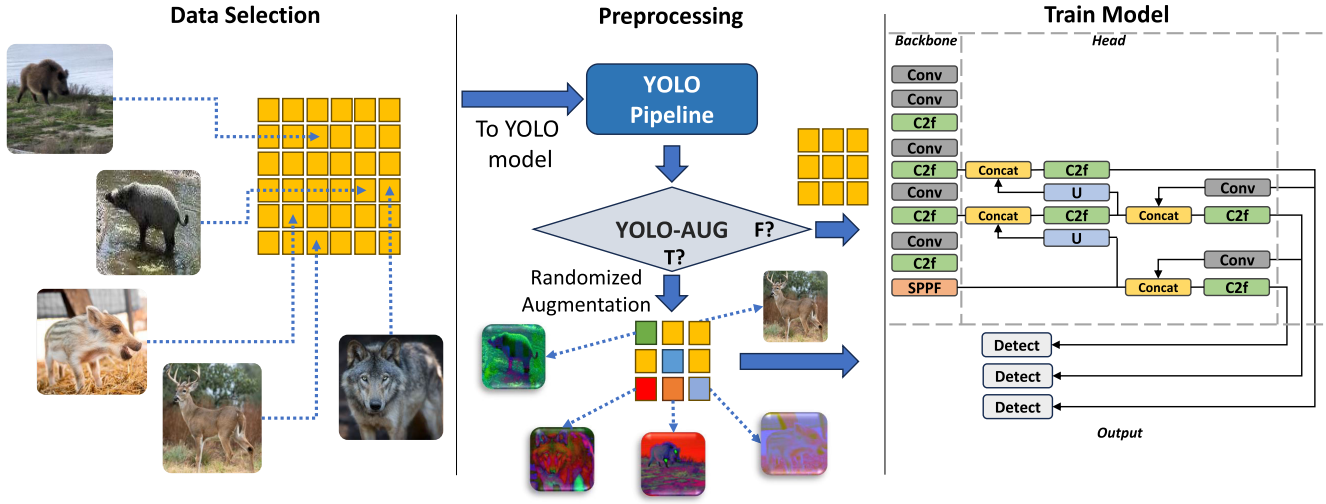


FIGURE 2. YOLO-AUG with data preprocessing (applied in all three training regimes).

is used as it is closest to human perception: L describes the lightness, while a and b encode opponent color axes (green–red, blue–yellow).

We also retain Ultralytics' standard augmentations such as Mosaic [16], horizontal flips, small rotations and random erasures to maximize diversity. Mosaic augmentation takes four images at random from the train set and combines randomly cropped parts of their contents into a synthetic picture that is then used during training. RandAugment [29] applies N randomly chosen operations of shared magnitude M per image, achieving performance on par with more complex AutoAugment methods with minimal overhead. Ultralytics provides RandAugment/AutoAugment-style policies for classification only, whereas the default detection pipeline uses augmentations such as mosaic/mixup/perspective and HSV-style color jitter rather than these policies. Our color-space module is therefore an additional component specific to detection training. Unless otherwise stated, YOLO-AUG is enabled in the YOLO-AUG (ON) runs. The YOLO-Baseline (OFF) runs disable only this module while keeping the remaining Ultralytics augmentations unchanged.

B. DATASET PREPARATION

The dataset is a refined version of that used in our previous work [26], comprising 4180 images (down from 4241) sourced from publicly licensed repositories [30], [31], [32]. Minor updates were conducted to correct instances with missing labels, and several corrupted files were removed. The main classes to detect are Wild Boar, Wolf and Deer. Other classes, such as the warthog, are included as hard negative, background examples with empty labels to improve generalization. The images also contain instances such as dogs, pigs, elephants and even humans. All of the aforementioned belong to the background class, and are represented by empty label files. The classes are also unbalanced, as seen in table 1, making stratification necessary for fair training[1]

The dataset for transfer learning is divided by two. Given the stratified nature of the original dataset, the division likewise should follow as close a distribution as possible.

[1]All imagery used for training is publicly licensed data. No sensitive personal data were used.

TABLE 1. Main dataset split.

	Split					
	Image Count			Instance Count		
Category	Train	Val	Test	Train	Val	Test
Boar	663	111	330	1115	156	548
Wolf	642	107	320	752	125	408
Deer	368	63	183	507	93	254
Background	836	140	417	-	-	-

In this method of learning, the images are divided from their corresponding training, validation and test folders. We have decided to create two subsets this way. One for the first training session, and the second for the transfer learning. In this scenario, all labels are kept for their corresponding subsets.

The dataset for semi-supervised learning has a similar approach to that of transfer learning, except for the second subset. While the division of images is exactly the same, for a fair comparative analysis the labels of the second subset, also known as pseudo-labels, are created by the first model, also known as the ''teacher'' model, which is trained on the first subset. The pseudo-labels are then assigned to the second subset, and the weights from the teacher model are transferred to the ''student'' model, where training continues. This method is a variation of the pseudo-labeling technique used in [33], [34] and [35].

C. K-FOLD CROSS-VALIDATION

We first assess the stability of YOLO-AUG (ON vs OFF) under repeated resampling via 5-fold cross-validation (CV) on the non-test pool. To avoid any data leakage, the fixed test split defined in Table 1 is created once and kept fully isolated. It is never used during CV, training, early stopping, confidence-threshold selection, or hyperparameter tuning. We perform 5-fold CV using an 80:20 split per fold *on the remaining non-test pool*, where 80% of the data goes to training and 20% to validation (held-out fold). We then continue by assessing the performance of YOLOv8n in terms of mAP50 and mAP50-95 scores, as well as Precision, Recall and Fitness.

This test has been conducted with YOLO-AUG based on the YOLOv8n model with an augmentation probability of 0.4, and 75 epochs were used to limit overfitting risk for a fair comparative analysis. These settings were fixed *a priori* for the CV study and were not tuned on the held-out test split. YOLO-AUG works by taking the images and automatically preprocessing them by applying randomly selected YOLO-AUG color-space transforms mentioned in Subsection III-A. Each image is heavily augmented in a different way in every epoch, which is a contributing factor to the model's robustness. The models are then evaluated on the unaugmented validation folds. Table 2 displays the Precision, Recall, mAP50, mAP50-95, and Fitness scores, along with their respective mean and standard deviation values across the five folds. Here, mAP50 denotes AP at an Intersection over Union (IoU) threshold of 0.50, while mAP50-95 denotes the mean AP over IoU thresholds 0.50:0.05:0.95 following the COCO evaluation protocol [2], [28].

TABLE 2. 5-Fold CV metrics of YOLO-AUG and baseline YOLO.

Metric	YOLO-AUG	YOLO-Baseline
	Mean ± Std. Dev.	Mean ± Std. Dev.
Precision	0.919 ± 0.01	0.915 ± 0.02
Recall	0.86 ± 0.02	0.859 ± 0.01
mAP50	0.925 ± 0.014	0.919 ± 0.012
mAP50-95	0.743 ± 0.012	0.738 ± 0.01
Fitness	0.761 ± 0.012	0.757 ± 0.01

TABLE 3. Statistical significance for mAP50 and mAP50-95 over 5-fold CV.

Metric	Δ	95% CI	p-value
		Lower – Upper	
mAP50	0.0058	0.0009 – 0.0095	0.0291
mAP50-95	0.0049	0.0007 – 0.0081	0.0295

The Fitness function is a metric used to evaluate the performance of the YOLO model based on its four preceding metrics. It is defined by the following equation:

$$F = w_P \cdot P + w_{\text{mAP}_{50}} \cdot mAP_{50} + w_R \cdot R + w_{\text{mAP}_{50-95}} \cdot mAP_{50-95} \quad (1)$$

With the mAP score equation as the following:

$$mAP = \frac{1}{N} \sum_{i=1}^{N} (AP_i) \quad (2)$$

We use the default Ultralytics YOLOv8 fitness weights as defined in the official implementation in [2]. We can see in Table 2 that, despite the inclusion of heavy augmentation without bloating the dataset, there is a slight improvement across all metrics, especially in mAP50 and mAP50-95 [28], where the model improved by 0.6 percentage points (pp) and 0.5 pp in performance respectively.

Because high point estimates alone do not guarantee generalization, we assess the stability of our gains on mAP50 and mAP50-95 using paired-difference statistics across the five CV folds. For each fold i, we compute the difference

$$d_i = \text{aug}_i - \text{base}_i,$$

which captures the improvement from our augmentations. From these five paired differences we derive the mean difference:

$$\bar{d} = \frac{1}{5} \sum_{i=1}^{5} d_i,$$

representing the average boost, and the standard error (SE):

$$SE = \frac{s_d}{\sqrt{5}},$$

Algorithm 1 Transfer Learning and Semi-Supervised Training

Require: Labeled data $D_L^{(1)}$, optional labeled data $D_L^{(2)}$, unlabeled data D_U
Require: Pretrained weights W_0, confidence threshold τ (and NMS IoU η if used)
Transfer Learning (TL):
Train on $D_L^{(1)}$ starting from W_0 to obtain W_1
if $D_L^{(2)}$ is provided **then**
    Fine-tune on $D_L^{(2)}$ starting from W_1 to obtain W_2
end if
Semi-Supervised Learning (SSL):
Train teacher on $D_L^{(1)}$ starting from W_0 to obtain W_T
Generate pseudo-labels $\hat{D}_U \leftarrow \text{Teacher}(D_U, \tau, \eta)$
Fine-tune student on $D_L^{(1)} \cup \hat{D}_U$ starting from W_T to obtain W_S

where s_d is the sample standard deviation of the d_i. We then construct a 95% confidence interval:

$$\bar{d} \pm t_{0.975,4}\, SE,$$

using the critical value $t_{0.975,4} \approx 2.776$ to indicate the range within which the true mean gain would lie 95% of the time under repeated sampling. Finally, we perform a paired two-tailed t-test to compute a p-value, defined as the probability of observing an average improvement at least as large as $\bar{d}$ if the true mean difference were zero. Table 3 summarizes these results: because both confidence intervals lie entirely above zero and the p-values fall below 0.05, we have evidence that YOLO-AUG provides a consistent improvement across CV folds. Precision and Recall, by contrast, showed high inter-fold variance with CIs crossing zero and p-values between 0.15 and 0.25. We report Precision/Recall across the 5 folds in Table 2, and summarize the corresponding statistical tests in Table 3. Final point estimates on the fixed held-out test split are reported later in the main results tables.

D. LEARNING TYPES

This section will cover the learning types that were used for experimentation, namely supervised learning, transfer learning and semi-supervised learning. For clarity, algorithm 1 summarizes the exact transfer-learning and semi-supervised procedures used in all experiments. The training-specific hyperparameters are then detailed in the subsections below.

1) SUPERVISED LEARNING

Supervised learning is training conducted on the full dataset with both YOLOv8n and YOLOv10n models. Each model is trained for 75 Epochs. The most notable hyperparameters, displayed in table 4, are the learning rates, with the implemented scheduler. In this case, linear scheduler is applied, defined in equation 3 based on the code in YOLOv8 pipeline:

$$\text{lr}(x) = \max\left(1 - \frac{x}{\text{E}}, 0\right) \times (1 - \text{lrf}) + \text{lrf} \tag{3}$$

Here, x is the current epoch the training is in, E is the total number of epochs for the training, and lrf is the annealing factor to reduce the learning rate linearly over time. The augmentations used during the training include Mosaic augmentation for a period equaling to half the number of epochs to first generalize, and then stabilize the training. Rotation of images have also been applied, and are randomly rotated in a range between −7 and 7 degrees. Random erasing is applied as well as horizontally flipped with probabilities of 0.4 and 0.5 respectively. Finally, we apply our own augmentation schemes using YOLO-AUG, further increasing robustness of the model and with a slight performance improvement on the test set compared to that of a training on images without heavy augmentation. The same parameter sets have been used here for comparison between the two models YOLOv8 and YOLOv10.

2) TRANSFER LEARNING

As explained in section III-B, in transfer learning, we take our dataset and divide it into two equally distributed subsets. The idea is to create model that is trained on the first subset, and then have its weights transferred to another model where the training would continue on the second subset. However, we would also like to explore that if, given the fine-tuning on the second subset is good enough, we could decrease the training time and reach a performance comparable to that of the full dataset supervised learning. This is where we introduce the additional hyperparameters. First, given that the data on the first subset is relatively limited, fine-tuning the model for convergence requires additional hyperparameters to be considered. For this reason, regularization parameters such as weight decay have been modified. We've also included cosine annealing [36], [37]. It periodically restarts the learning rate, which aids in escaping local minima and saddle points, thus resulting in higher convergence and performance in object detection in this case. It is defined as:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{T_{cur}}{T_{max}}\pi\right)\right) \tag{4}$$

where η_t is the learning rate at the current time step, $\eta_{\min}$ and $\eta_{\max}$ are the minimum and maximum learning rates, respectively, T_{cur} is the number of epochs since the last restart, and T_{max} is the maximum number of epochs for the training session.

We then consider the number of epochs and the fine-tuning of the transfer learning hyperparameters. As the weights are being transferred, it means that learning cannot continue at the same pace. Thus, as the optimizer (AdamW [38] in our case) learning rate scheduling is reset, the starting epochs would not have as adverse an impact on the transferred weights as they would with larger learning rates. Stochastic Gradient Descent (SGD) was also considered and tested

here, but ultimately AdamW performed better after numerous trials.

3) SEMI-SUPERVISED LEARNING

The dataset split follows the same two-subset structure used in transfer learning. However, in the semi-supervised setting the second subset is initially unlabeled. We first train a teacher model on the labeled subset (Subset 1), using the same base training procedure as in transfer learning. We then use this teacher to generate pseudo-labels for the images in the unlabeled subset (Subset 2). As pseudo-labels can contain errors and noisy bounding-box coordinates, the student model is trained on a combined set that includes both the manually labeled data and the pseudo-labeled data, i.e., $D_L \cup \hat{D}_U$, and we adjust additional hyperparameters accordingly. In particular, we enable cosine learning-rate scheduling and tune the loss-component weights (classification, box regression, and Distribution Focal Loss) used in the Ultralytics YOLOv8 objective [2].

Distribution Focal Loss (DFL), introduced as part of the generalized focal loss formulation for distributional bounding-box localization [39] and later leveraged for localization-quality estimation using distribution statistics in Generalized Focal Loss (GFLv2) [40], models bounding-box regression by predicting a discrete probability distribution over each box coordinate (or offset), rather than regressing a single deterministic value. This distributional formulation captures ambiguity and uncertainty in localization and enables more precise boundary refinement by encouraging high probability mass near the ground-truth target. In contrast, Focal Loss [41] reshapes cross-entropy to mitigate the extreme foreground–background class imbalance in dense detectors by down-weighting easy negatives and focusing learning on hard examples. Therefore, while Focal Loss primarily targets classification imbalance, DFL targets spatial localization quality via distributional regression. In the YOLO pipeline, DFL is implemented as follows:

- $\mathbf{P}_{dist}$: Represents the predicted probability distribution over the bins for each bounding box coordinate.
- $\mathbf{T}_{coord}$: Represents the ground truth coordinates for the bounding box in continuous form (i.e., real-valued coordinates), which is subsequently mapped to discrete bins for loss computation.

$$\mathbf{T}_{coord} = \text{clamp}(\mathbf{T}_{coord}, 0, N_{bins} - 1 - \epsilon) \tag{5}$$

- $\mathbf{B}_{left}$: Indicates the left bin or the integer part of the discretized ground truth coordinate, aligning with the lower bin for interpolation.

$$\mathbf{B}_{left} = \lfloor \mathbf{T}_{coord} \rfloor \tag{6}$$

- $\mathbf{B}_{right}$: Represents the right bin or the next higher bin for interpolation.

$$\mathbf{B}_{right} = \lfloor \mathbf{B}_{left} \rfloor + 1 \tag{7}$$

- $\mathbf{W}_{left}$: Represents how much of the coordinate value falls within the left bin, based on its fractional position.

$$\mathbf{W}_{left} = \mathbf{B}_{right} - \mathbf{T}_{coord} \tag{8}$$

- $\mathbf{W}_{right}$: Represents how much of the coordinate value falls within the right bin, based on its fractional position.

$$\mathbf{W}_{right} = 1 - \mathbf{W}_{left} \tag{9}$$

- N_{bins}: Represents the total number of discrete bins used to approximate the continuous coordinate range. In YOLOv8, this is a constant equal to 16.
- CE: Acronym for Cross-Entropy loss.

The DFL equation is then defined as:

$$\mathcal{L}_{DFL} = \frac{1}{N}\sum_{i=1}^{N}\Big[\mathbf{W}_{left}\cdot\mathcal{L}_{CE}(\mathbf{P}_{dist},\mathbf{B}_{left}) + \mathbf{W}_{right}\cdot\mathcal{L}_{CE}(\mathbf{P}_{dist},\mathbf{B}_{right})\Big] \tag{10}$$

The following terms are defined for the calculation of both classification, bounding box and total losses:

- N: Number of target instances
- $\mathbf{S}_{pred}$: Predicted scores
- $\mathbf{S}_{target}$: Target scores
- $\mathcal{L}_{BCE}$: Binary Cross Entropy Loss
- $\mathbf{B}_{pred}$: Predicted bounding boxes
- $\mathbf{B}_{target}$: Target bounding boxes
- $\mathbf{B}_{size}$: Batch size
- $\mathbf{W}$: Weight for each bounding box
- **IoU**: Intersection over Union
- α, β, γ: Weight factors for bounding box loss, classification loss and distributional focal loss respectively.

The classification loss and bounding box loss equation are then respectively defined as:

$$\mathcal{L}_{cls} = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}_{BCE}(\mathbf{S}_{pred},\mathbf{S}_{target}) \tag{11}$$

$$\mathcal{L}_{bbox} = \frac{1}{N}\sum_{i=1}^{N}(1-\mathbf{IoU}(\mathbf{B}_{pred},\mathbf{B}_{target}))\cdot\mathbf{W} \tag{12}$$

The total loss is calculated as:

$$\mathcal{L}_{total} = \mathcal{L}_{bbox}\cdot\alpha + \mathcal{L}_{cls}\cdot\beta + \mathcal{L}_{DFL}\cdot\gamma \tag{13}$$

Ultimately, the final loss is calculated as:

$$\mathcal{L}_{output} = \mathcal{L}_{total}\cdot\mathbf{B}_{size} \tag{14}$$

IV. EXPERIMENTATION, RESULTS, POWER AND MEMORY CONSIDERATIONS

This section focuses on the learning aspects of the YOLO models using YOLOv8n and YOLOv10n with pretrained weights by training them fully on the dataset via supervised learning, employing transfer learning, and employing semi-supervised learning. We present the hyperparameters used for all training sessions in Table 4. All final metrics are reported

on the fixed held-out test split in Table 1, which is fully isolated from cross-validation and any tuning (Section III-C).

All experiments conducted here are on an ACER Nitro 5 AN517-55 laptop equipped with an RTX 3050 4GB GDDR6 GPU, 16 GB RAM, and an Alder Lake-P Intel Core i5-12500H processor (12 cores, 16 threads). Hardware comparison in terms of training time and power consumption is conducted between the RTX 3050 and the NVIDIA Jetson Orin Nano 8GB Development Kit GPUs. All experiments were run in Python 3.10.8 using Ultralytics v8.3.32 and PyTorch v2.5.1 with CUDA 11.8 support (torchvision v0.20.1 and torchaudio v2.5.1). OpenCV v4.8.0.76 and NumPy v1.23.5 were used for image I/O and preprocessing. On the Jetson Orin Nano, the system software was NVIDIA JetPack 6.2.1 (L4T/Jetson Linux 36.4.4), while the RTX 3050 laptop ran Windows 10 (22H2). For Jetson measurements, the device was configured in the 15 W power mode (nvpmodel) with default clocks. Training was executed with automatic mixed precision (AMP) enabled.

To isolate the effect of the proposed augmentation, each experiment is run in two conditions: YOLO-AUG (ON) and YOLO-Baseline (OFF). The OFF condition disables only the proposed multi–color-space module while keeping all other Ultralytics augmentations and training hyperparameters unchanged (Section III-A). Training time is reported as mean $\pm$ standard deviation over three independent runs with different random seeds, under the same configuration and fixed dataset split; timing is obtained from the training pipeline's end-to-end wall-clock duration per run. Due to non-determinism in modern GPU training stacks, small run-to-run variation is expected even under identical configurations. For transfer learning, the reported time includes both Subset 1 training and Subset 2 fine-tuning. For semi-supervised learning, the reported time includes teacher training and student fine-tuning. Pseudo-label generation overhead is reported separately when applicable.

Supervised Learning Training and Results:

We begin by running a supervised learning session with each of the YOLOv8n and YOLOv10n models for a comparative analysis of their metrics and overall performance. We use 75 epochs each, using a batch size of 128, a start and final learning rate of 0.001 and 0.0075 respectively, and a weight decay of 0.0005. To isolate the effect of the proposed augmentation, each supervised experiment is run in two conditions: YOLO-Baseline (OFF) and YOLO-AUG (ON). In the OFF condition, the YOLO-AUG module is disabled, while all other Ultralytics augmentations and hyperparameters remain unchanged. In the ON condition, YOLO-AUG is enabled and is applied with probability 0.4, consistent with the k-Fold CV setting in III-C. Training time is reported as mean $\pm$ standard deviation over 3 repeated runs (different seeds) with the same configuration and fixed dataset split.

In the YOLO-Baseline (OFF) condition (Table 5), YOLOv8n achieves an F1-score of 0.880 compared to YOLOv10n's 0.860, indicating stronger object detection and class assignment capabilities with YOLOv8n. YOLOv8n also attains mAP50 and mAP50-95 scores of 0.915 and 0.730 respectively, compared to YOLOv10n's 0.896 and 0.707, indicating improved localization performance. The total training time for YOLOv8n is 21.27 $\pm$ 0.73 minutes, compared to YOLOv10n's 24.56 $\pm$ 0.15 minutes, confirming that YOLOv8n is faster to train under identical supervised settings.

In the YOLO-AUG (ON) condition (Table 6), both models show consistent gains in detection performance relative to their OFF counterparts. YOLOv8n improves to an F1-score of 0.890, with mAP50 and mAP50-95 increasing to 0.922 and 0.735 respectively, while YOLOv10n improves to an F1-score of 0.870, with mAP50 and mAP50-95 increasing to 0.901 and 0.712. In terms of timing, YOLOv8n trains in 20.96 $\pm$ 0.15 minutes, while YOLOv10n trains in 25.53 $\pm$ 0.76 minutes, indicating that the end-to-end training-time impact of enabling YOLO-AUG is small relative to the observed run-to-run variance, and can differ across models. Across both OFF and ON conditions, YOLOv8n remains superior to YOLOv10n in our setup in terms of detection performance and training speed.

We additionally include training on the Jetson Orin Nano and compare it with the RTX 3050 to quantify the training-time gap between representative far-edge and laptop GPU hardware. The results are reported in Table 7. Across both YOLO-AUG (ON) and YOLO-Baseline (OFF), the Jetson requires roughly 3 times longer training time than the RTX 3050 for both YOLOv8n and YOLOv10n, and the relative ordering between models is preserved across platforms (YOLOv10n consistently trains longer than YOLOv8n). Table 7 also reports the Jetson-side augmentation overhead (Δ_J), which is modest in the supervised setting. Enabling YOLO-AUG increases Jetson training time by 4.91 min for YOLOv8n and 5.66 min for YOLOv10n (around 7.94% in both cases).

Transfer Learning Training and Results:

The first stage trains on Subset 1 (50% of the dataset) and is shared as the starting point for both TL and SSL. We then continue training on Subset 2 using either true labels (TL) or pseudo-labels (SSL). In the second subset, we have lowered the starting and final learning rates to 0.00025 and 0.00075 respectively given that it is the same in size as the first subset, which would mean that the model will easily overfit on the second subset, making it forget its previously attained knowledge, a concept also known as catastrophic forgetting. This is also an issue if the number of epochs is too high, so a lower number of epochs is also necessary. In this case, it is reduced from 75 on the first subset to 40 in the training on second subset, and we have enabled cosine annealing as explained in III-D2.

To isolate the contribution of YOLO-AUG, we report TL results in both YOLO-Baseline (OFF) and YOLO-AUG (ON) conditions, under identical training settings where the OFF condition disables only the proposed module.

TABLE 4. Hyperparameters used for YOLOv8 and YOLOv10 models.

Hyperparameters	Values										Description
	YOLOv8					YOLOv10					YOLO model used
	Full Dataset	Transfer Learning		Semi-Supervised Learning		Full Dataset	Transfer Learning		Semi-Supervised Learning		Training type run
	-	1st Subset	2nd Subset	1st Subset	2nd Subset	-	1st Subset	2nd Subset	1st Subset	2nd Subset	Defines the set or subset used for training
Epochs	75	75	40	75	50	75	75	40	75	50	The number of epochs for the training
Batch Size	128	128	128	128	128	128	128	128	128	128	The number of images considered at the same time. It defines the number of iterations per Epoch
Initial LR	0.001	0.001	0.00025	0.001	0.0001	0.001	0.001	0.00025	0.001	0.0001	The learning rate at which the training starts
LR Factor at end of schedule	0.0075	0.0075	0.00075	0.0075	0.0008	0.0075	0.0075	0.00075	0.0075	0.0008	The final learning rate the training reaches. It is affected by schedulers
Weight Decay	0.0005	0.0005	0.00025	0.0005	0.0005	0.0005	0.0005	0.00025	0.0005	0.0005	A regularization term, also known as L2 regularization, used for penalizing large weights to prevent overfitting
Cosine Annealing	False	False	True	False	True	False	False	True	False	True	A learning rate scheduler that begins with a high learning rate and quickly reduces it to a low number before quickly increasing it again.
Box Loss	7.5	7.5	7.5	7.5	6.5	7.5	7.5	7.5	7.5	6.5	Weight of the bounding box loss, which measures how well the predicted bounding box matches the ground truth.
Classification Loss	0.5	0.5	0.5	0.5	1	0.5	0.5	0.5	0.5	1	Weight of the classification loss, determining how important it is for the model to correctly classify detected objects into the right category.
Distributional Focal Loss	1.5	1.5	1.5	1.5	7	1.5	1.5	1.5	1.5	7	Weight of the distribution focal loss, helping the model make finer adjustments to its predictions by learning from the distribution of possible outputs, improving accuracy.
Aug. Prob. (ours)	0.4	0.4	0.4	0.4	0.5	0.4	0.4	0.4	0.4	0.5	Probability that a random heavy augmentation is applied on the image in question.

TABLE 5. Learning metrics for YOLOv8n and YOLOv10n models (baseline).

YOLO Model	Learning Type	1st Subset (50% of Dataset)				2nd Subset (50% of Dataset)				Complexity Reduction	
		mAP50	mAP50-95	F1-Score	Epochs	mAP50(.)	mAP50-95(.)	F1-Score(.)	Epochs(.)	Avg. Epochs	Training Time ± std(min)
YOLOv8n	Supervised	0.915	0.730	0.880	75 (Full set)	N/A	N/A	N/A	N/A	75 (0%)	21.27 ± 0.73 (0%)
	Transfer	0.863	0.667	0.820	75	0.886(+2.3%)	0.681(+1.4%)	0.860(+4%)	40	57.5 (-23.33%)	14.76 ± 0.31 (-30.61%)
	Semi-Supervised	0.863	0.667	0.820	75	0.869(+0.6%)	0.642(-2.5%)	0.850(+3%)	50	62.5 (-16.67%)	23.65 ± 0.61 (+11.19%)
YOLOv10n	Supervised	0.896	0.707	0.860	75 (Full set)	N/A	N/A	N/A	N/A	75 (0%)	24.56 ± 0.15 (0%)
	Transfer	0.834	0.638	0.800	75	0.847(+1.3%)	0.660(+2.2%)	0.800(+0%)	40	57.5 (-23.33%)	17.39 ± 0.59 (-29.19%)
	Semi-Supervised	0.834	0.638	0.800	75	0.819(-1.5%)	0.610(-2.8%)	0.780(-2%)	50	62.5 (-16.67%)	27.84 ± 0.55 (+13.36%)

TABLE 6. Learning Metrics for YOLOv8n and YOLOv10n Models (YOLO-AUG).

YOLO Model	Learning Type	1st Subset (50% of Dataset)				2nd Subset (50% of Dataset)				Complexity Reduction	
		mAP50	mAP50-95	F1-Score	Epochs	mAP50(.)	mAP50-95(.)	F1-Score(.)	Epochs(.)	Avg. Epochs	Training Time ± std (min)
YOLOv8n	Supervised	0.922	0.735	0.890	75 (Full set)	N/A	N/A	N/A	N/A	75 (0%)	20.96 ± 0.15 (0%)
	Transfer	0.881	0.673	0.860	75	0.901(+2.0%)	0.692(+1.9%)	0.880(+2%)	40	57.5 (-23.33%)	15.46 ± 0.12 (-26.24%)
	Semi-Supervised	0.881	0.673	0.860	75	0.889(+0.8%)	0.668(-0.5%)	0.870(+1%)	50	62.5 (-16.67%)	25.70 ± 0.35 (+22.61%)
YOLOv10n	Supervised	0.901	0.712	0.870	75 (Full set)	N/A	N/A	N/A	N/A	75 (0%)	25.53 ± 0.76 (0%)
	Transfer	0.858	0.661	0.800	75	0.853(-0.5%)	0.667(+0.6%)	0.820(+2%)	40	57.5 (-23.33%)	18.05 ± 0.41 (-29.30%)
	Semi-Supervised	0.858	0.661	0.800	75	0.825(-3.3%)	0.620(-4.1%)	0.790(-1%)	50	62.5 (-16.67%)	29.02 ± 0.41 (+13.67%)

In the YOLO-Baseline (OFF) condition (Table 5), YOLOv8n improves from Subset 1 to Subset 2 by +2.3% in mAP50 (0.863 to 0.886), +1.4% in mAP50-95 (0.667 to 0.681), and +4% in F1-score (0.820 to 0.860). The overall average epochs drop from 75 to 57.5 (-23.33%), and the end-to-end training time decreases from 21.27 to 14.76 minutes (−30.61%), showing that TL reduces training overhead substantially while retaining competitive performance.

In the YOLO-AUG (ON) condition (Table 6), YOLOv8n similarly improves from Subset 1 to Subset 2 by +2% in mAP50 (0.881 to 0.901), +1.9% in mAP50-95 (0.673 to 0.692), and +2% in F1-score (0.860 to 0.880), while maintaining the same complexity reduction (57.5 average epochs). The training time is reduced from 20.96 to 15.46 minutes (-26.24%), indicating that TL remains the dominant contributor to time reduction, with YOLO-AUG providing consistent metric gains on top of the same TL schedule.

For YOLOv10n, the same TL schedule is applied. In the YOLO-Baseline (OFF) condition, Subset 2 yields 0.847 mAP50, 0.660 mAP50-95 and 0.800 F1-score, while in the YOLO-AUG (ON) condition (Table 6) Subset 2 yields 0.853 mAP50, 0.667 mAP50-95 and 0.820 F1-score. Across both OFF and ON conditions, YOLOv8n remains stronger than YOLOv10n under TL in our setup, both in detection performance and in end-to-end training time (Tables 5 and 6, Fig. 3).

TABLE 7. Supervised training time (minutes) with/without YOLO-AUG on RTX 3050 and Jetson Orin Nano (mean over 3 runs).

Model	RTX OFF	RTX ON	Jet OFF	Jet ON	ΔJ	J(%)	J/R (ON)
YOLOv8n	21.27	20.96	61.81	66.72	4.91	7.94	3.183
YOLOv10n	24.56	25.53	71.26	76.92	5.66	7.94	3.013

Semi-Supervised Learning Training and Results:

The hyperparameters in semi-supervised learning are fine-tuned to accommodate the noise in the labels generated by pseudo-labeling. It is important to note that the pseudo-labels for the second subset are generated by running the teacher model on the unlabeled images and keeping predictions with confidence greater than or equal to τ_{conf} (in our experiments, $\tau_{\text{conf}} = 0.5$), using an NMS IoU threshold of 0.6. In this case, the number of epochs is 50 for the second subset, and the starting and final learning rates are 0.0001 and 0.0008, respectively. The weights of the losses have been modified as well. The box loss weight α is reduced to 6.5, which helped to offset inaccuracies by slightly

lessening the penalty associated with misaligned bounding boxes. This allowed the model to generalize better over the noisy pseudo-labeled boxes. The classification loss β is increased to 1.0 to emphasize the correct class label for each bounding box, mainly due to the fact that some pseudo-labels may be misclassified. As for the distributional focal loss weight γ, it is increased to 7. As previously stated, DFL is used to refine boundaries and increase the precision of localizing objects, and given the noisy pseudo-labels of semi-supervised learning, it would have made sense to decrease this weight. Counterintuitively, however, after numerous fine-tuning attempts, increasing it to 7 gave us the best results, with values of γ under 1.5 giving marginally lower metrics in comparison.

To isolate the contribution of YOLO-AUG, we report SSL results in both YOLO-Baseline (OFF) and YOLO-AUG (ON) conditions, under identical training settings where the OFF condition disables only the proposed module.

In the YOLO-Baseline (OFF) condition (Table 5), YOLOv8n improves from Subset 1 to Subset 2 by +0.6% in mAP50 (0.863 to 0.869) and +3% in F1-score (0.820 to 0.850), while mAP50-95 decreases by -2.5% (0.667 to 0.642), reflecting the greater localization sensitivity at higher IoU thresholds under noisy pseudo-labels. In contrast to earlier timing observations, the end-to-end SSL training time is higher than supervised learning in this final configuration: 23.65 ± 0.61 minutes for SSL versus 21.27 ± 0.73 minutes for supervised learning (+11.19%).

In the YOLO-AUG (ON) condition (Table 6), YOLOv8n achieves higher SSL performance than the OFF condition, reaching 0.889 mAP50, 0.668 mAP50-95 and 0.870 F1-score on Subset 2. However, the training time is also higher: 25.70 ± 0.35 minutes for SSL versus 20.96 ± 0.15 minutes for supervised learning (+22.61%). Therefore, in our setup SSL primarily provides automation benefits (reduced manual labeling) rather than a reduction in training time.

For YOLOv10n, the SSL teacher produces noisier pseudo-labels in our experiments, and performance drops from Subset 1 to Subset 2 in both conditions (Tables 5 and 6). In the OFF condition, mAP50 decreases from 0.834 to 0.819 and mAP50-95 decreases from 0.638 to 0.610. In the ON condition, YOLO-AUG improves the final Subset 2 SSL results slightly (0.825 mAP50 and 0.620 mAP50-95), but YOLOv10n remains below YOLOv8n in SSL performance, while also requiring longer end-to-end training time (Tables 5 and 6).

A. MEMORY CONSIDERATIONS

Aside from benchmarking both models, we have considered not only the memory consumption of YOLOv8n and YOLOv10n, but how they behave on a continuous time basis. Interestingly, we have found that the reported memory consumptions do not necessarily accurately reflect the ones shown during the training. In particular, there are patterns in memory spikes that reoccur on a quasi-periodic basis during training, and in particular during epoch changes. In addition to the memory consumption being relative to the batch size used as well as the image resizing factor considered, each model has its own memory consumption even when having the exact same hyperparameters pertaining to memory consumption during training.

In our case, the batch size that we have used is 128 throughout all of our experiments, and all pictures were of size 256×256 pixels. They are pictures that had been automatically resized, and then inserted into training. We have observed that during training, periodic evaluation phases (as configured in the training pipeline) coincide with transient memory spikes to assess the current model's performance per epoch as its metrics get updated. After empirical testing, we have concluded that the memory spike is a factor of the maximum memory consumption of the epoch in question, and is empirically defined as:

$$M_{\text{spike}}(i) \approx 1.3 \cdot M_{\text{epoch}}(i) \tag{15}$$

Here, $M_{\text{spike}}(i)$ is the memory spike observed during the validation phase after epoch i, $M_{\text{epoch}}(i)$ is the memory consumption during the training phase of epoch i, and 1.3 is the empirical factor indicating the proportional increase in memory usage during validation. Please note that the factor is not always the same, and sometimes can be lower. The only reason we've included this factor is as a safety margin, as it was the highest ratio observed during our experiments, rather than a strict rule that always holds. As for the monitored results, they are found in figure 4 for both YOLOv8n and YOLOv10n. All measurements were taken in real time with 100 milliseconds of sampling frequency.

For instance, we can see in figure 4 for YOLOv8n, the memory consumption spikes from 2.8 GB to 3.3 GB. Likewise, for YOLOv10n, the memory spikes from 3.5 GB to 4 GB plus 0.2 GB borrowed from shared memory (system RAM). We can infer that YOLOv8n is more memory efficient compared to YOLOv10n. Additionally, it gives us an idea of the ideal batch size as a hyperparameter based on the image dimensions. This is important if we are considering deployable GPUs such as the Jetson Orin Nano, which is equipped with 8 GB of shared memory between GPU and Central Processing Unit (CPU), as it helps us avoid possible Out-Of-Memory (OOM) errors. As a rough rule of thumb in this work, in the case of shared memory, we assume 50% to be effectively available to the GPU and 50% to the CPU, meaning that we have about 4 GB of memory dedicated to the GPU. For instance, considering the 1.3 safety factor and image sizes of 256 × 256 pixels, the batch size that could be considered would be equal to 160 for YOLOv8n, but for YOLOv10n it would be 120.

B. POWER CONSUMPTION

Power consumption of far-edge devices is not limited to the hardware being utilized, but also the amount of processing required to run the applications in question. In this case, we benchmark the power consumption of YOLOv8n and YOLOv10n to determine which model is more cost-effective

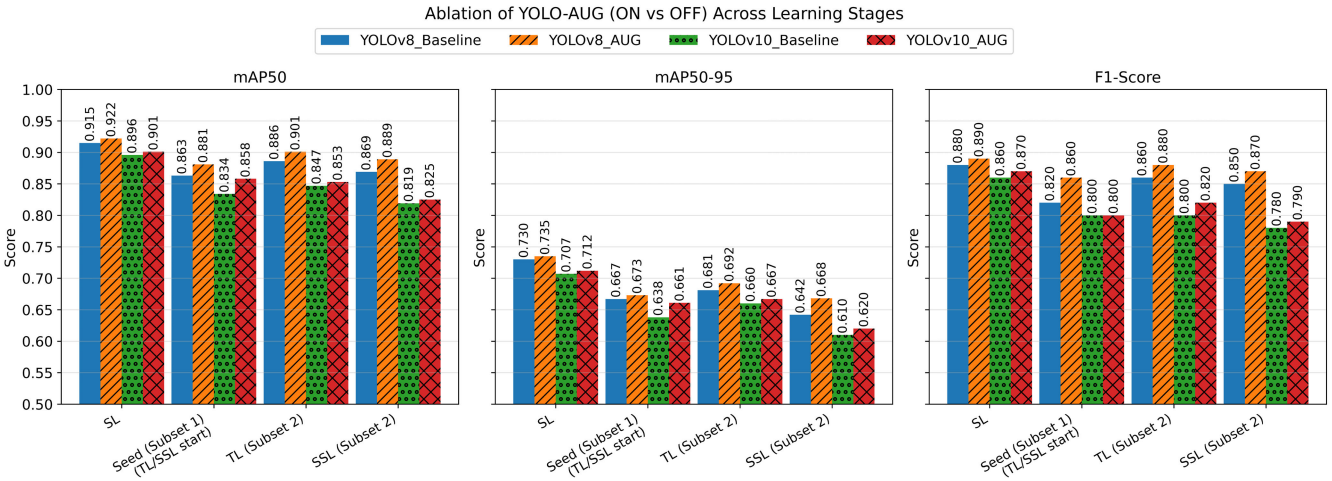


FIGURE 3. Ablation of YOLO-AUG on YOLOv8n vs YOLOv10n metrics.

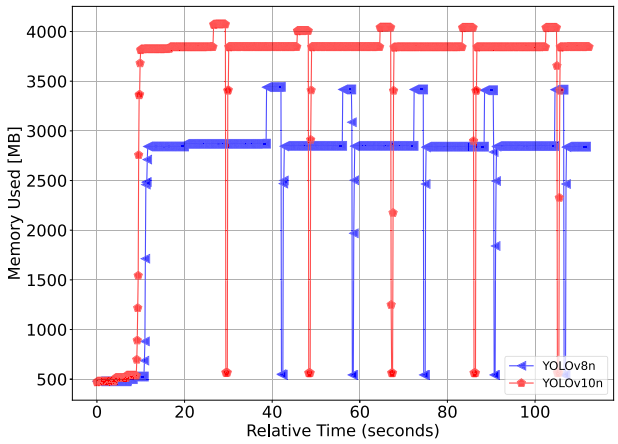


FIGURE 4. YOLOv8n vs YOLOv10n memory usage during training.

during training. We consider the supervised learning setting under YOLO-AUG (ON), and the results are plotted in Fig. 5.

We report Jetson Orin Nano power measurements only for the YOLO-AUG (ON) configuration. This choice was made because YOLO-AUG represents the worst-case end-to-end training workload among our settings (slightly longer wall-clock training time), and therefore provides a conservative estimate for energy budgeting in far-edge retraining. We did not record a separate Jetson power trace for YOLO-Baseline (OFF). However, since training power was observed to be approximately steady during supervised runs, we estimate YOLO-Baseline (OFF) session energy by combining the measured average `VDD_IN` power with the corresponding OFF training time. On the RTX 3050, power traces were collected during training and are used here primarily as a desktop reference baseline. On the Jetson Orin Nano, power was sampled from tegrastats power telemetry at a 100 ms interval (using the tool's interval setting). In our logs, the reported Jetson power corresponds to `VDD_IN`, i.e., the total module input power (CPU+GPU+ State of Charge (SoC)), rather than GPU-only power [42]. On the RTX 3050,

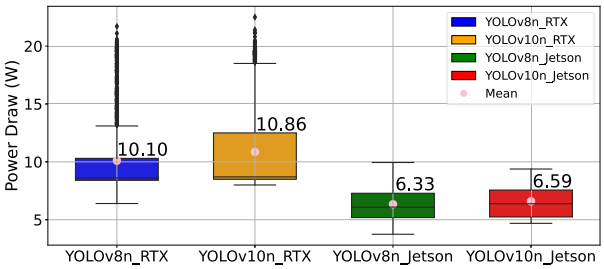


FIGURE 5. YOLOv8n vs YOLOv10n power usage during supervised training under YOLO-AUG (ON).

GPU power was sampled using TechPowerUp GPU-Z sensor telemetry with logging enabled. All reported values refer to on-device telemetry during training, rather than total system power measured at the wall (and thus exclude PSU/battery conversion losses and peripherals).

The boxplot in Fig. 5 indicates that YOLOv10n exhibits higher variability in training power draw than YOLOv8n (larger interquartile range and more extreme values), consistent with workload-dependent fluctuations during training. On the RTX 3050, the mean training power is 10.1 W for YOLOv8n and 10.86 W for YOLOv10n. We report the RTX 3050 results as a desktop baseline, while field feasibility is assessed using the Jetson Orin Nano measurements.

Combining the measured average power with the supervised training times in Table 7, the estimated energy per 75-epoch training session is computed as $E[\text{Wh}] = P[\text{W}] \cdot t[\text{min}]/60$. On the RTX 3050, this yields approximately 3.53 Wh (YOLOv8n) and 4.62 Wh (YOLOv10n) under YOLO-AUG (ON), and 3.58 Wh and 4.45 Wh under YOLO-Baseline (OFF). On the Jetson Orin Nano, we report the average `VDD_IN` training power (total module input) obtained via `tegrastats`, which is 6.33 W (YOLOv8n) and 6.59 W (YOLOv10n). This yields approximately 7.04 Wh (YOLOv8n) and 8.45 Wh (YOLOv10n) per training session with YOLO-AUG (ON), and an estimated 6.52 Wh and 7.83 Wh with YOLO-Baseline (OFF), respectively. The

additional energy attributable to enabling YOLO-AUG on the Jetson in the supervised setting is therefore modest (0.52-0.62 Wh per session in our runs).

1) FIELD FEASIBILITY AND POWER PROVISIONING

The prototype far-edge node considered in this work is powered by a custom Li-ion battery pack configured as 4-Series 6-Parallel (4S6P) using 2650 mAh cells, equipped with a Battery Management System (BMS) for charging and balancing, and a DC buck converter with measured efficiency of approximately $\eta = 0.9$. Assuming a nominal cell voltage of 3.7 V, the pack energy is approximately 235 Wh, corresponding to roughly 212 Wh delivered after conversion losses, excluding depth-of-discharge limits. Under the estimated Jetson Orin Nano training energy of 7.04 Wh (YOLOv8n) and 8.45 Wh (YOLOv10n) per session, this corresponds to on the order of $\sim$30 and $\sim$25 training sessions per full charge, respectively (depending on allowable depth of discharge and concurrent workload). The nominal pack voltage for a 4S configuration at 3.7 V is 14.8 V.

Using the measured average training power on the Jetson Orin Nano, we can also estimate the corresponding average current drawn from the battery pack. Assuming a nominal 4S pack voltage of $V_{\text{pack}} \approx 14.8$ V and DC conversion efficiency $\eta = 0.9$, the average pack current during training is approximately $I_{\text{pack}} \approx P/(V_{\text{pack}}\eta) \approx 0.48$ A (YOLOv8n, P=6.33 W) and 0.50 A (YOLOv10n, P=6.59 W). For a 4S6P configuration, this corresponds to roughly $I_{\text{cell}} \approx I_{\text{pack}}/6 \approx 0.08$ A per cell on average, i.e., a C-rate on the order of $\sim$0.03C for 2650 mAh cells. C-rate is commonly defined as discharge current normalized by rated capacity (e.g., 1C discharges in $\sim$1 hour).

Unlike inference workloads that can be duty-cycled between triggers, training is sustained computation. Therefore, we assume infrequent retraining scheduled only when adequate energy is available. In practice, retraining is intended to be periodic or opportunistic (e.g., after accumulating enough images or detecting performance drift). When energy availability is insufficient, an alternative is to transmit a subset of samples for remote training and then deploy updated weights to the edge node. The battery system is compatible with solar charging, but the latter was not evaluated in this study.

V. CONCLUSION

In this work, we investigated the feasibility of training YOLO-based detectors in far-edge settings under limited computational resources and practical time constraints. We introduced YOLO-AUG, an on-the-fly multi-color-space augmentation module integrated into the Ultralytics training pipeline, and evaluated it through controlled comparisons between YOLO-AUG (ON) and a YOLO-Baseline (OFF) condition where only the proposed module is disabled while all other Ultralytics augmentations and training settings remain unchanged. Across our experiments, YOLO-AUG improves robustness to appearance variation and provides modest but consistent gains in detection performance, supported by both the fixed held-out test split and auxiliary k-fold cross-validation analysis.

We also performed a comparative analysis between YOLOv8n and YOLOv10n under identical learning regimes and configurations. Overall, YOLOv8n was better suited for our deployment-oriented setting, achieving stronger detection performance and more favorable training-time behavior in our experiments. Focusing on YOLOv8n, we then characterized the trade-offs between supervised, transfer, and semi-supervised training. Supervised learning yields the strongest performance, while transfer learning achieves near-comparable metrics with substantially reduced training time by fine-tuning on a second subset. Semi-supervised learning provides a more automated workflow by reducing manual labeling through pseudo-labeling, with a trade-off in performance due to noisy pseudo-labels. Finally, we assessed memory and power considerations relevant to far-edge retraining feasibility. We reported GPU memory behavior over time (including transient spikes) and provided practical guidance for avoiding OOM failures by selecting batch size under constrained memory budgets. For power/energy budgeting on the Jetson Orin Nano, we report the average `VDD_IN` training power (total module input) obtained via `tegrastats` during the supervised training runs. Using the ON/OFF training times in Table 7 together with the measured average power, we estimate the corresponding energy per training session and bound the additional session-level energy attributable to enabling YOLO-AUG, thus providing a budgeting reference for occasional retraining under battery operation, where the differentiator between ON/OFF energy is the additional training time. These conclusions are drawn from a single curated dataset compiled from multiple publicly licensed repositories, focused on wild boar, wolf, and deer. As such, the results should be interpreted as evidence of the effectiveness of YOLO-AUG and the compared far-edge training strategies in this specific context. Evaluating the pipeline on additional camera-trap datasets, additional species, and broader far-edge hardware configurations is left as future work.

REFERENCES

[1] P. Vajas, E. Von Essen, L. Tickle, and M. Gamelon, "Meeting the challenges of wild boar hunting in a modern society: The case of France," *Ambio*, vol. 52, no. 8, pp. 1359–1372, Aug. 2023. [Online]. Available: https://link.springer.com/article/10.1007/s13280-023-01852-1

[2] Ultralytics. (2025). *Ultralytics Github Repository*. Accessed: Jul. 24, 2025. [Online]. Available: https://github.com/ultralytics

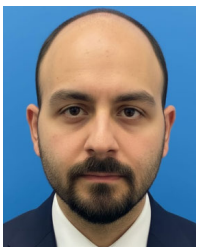
[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," 2015, *arXiv:1506.02640*.

[4] A. A. Murat and M. S. Kiran, "A comprehensive review on YOLO versions for object detection," *Eng. Sci. Technol., Int. J.*, vol. 70, Oct. 2025, Art. no. 102161.

[5] M. Hammami, D. Friboulet, and R. Kechichian, "Cycle GAN-based data augmentation for multi-organ detection in CT images via YOLO," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Abu Dhabi, United Arab Emirates, Oct. 2020, pp. 390–393.

[6] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," 2017, *arXiv:1703.10593*.

[7] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
[8] S.-Y. Lin and H.-Y. Li, "Integrated circuit board object detection and image augmentation fusion model based on YOLO," *Frontiers Neurorobotics*, vol. 15, Nov. 2021, Art. no. 762702, doi: 10.3389/fnbot.2021.762702.
[9] X. Sun, X. Jia, Y. Liang, M. Wang, and X. Chi, "A defect detection method for a boiler inner wall based on an improved YOLO-v5 network and data augmentation technologies," *IEEE Access*, vol. 10, pp. 93845–93853, 2022.
[10] O. Mohan Krishnan, P. R. Kalyan, M. Ilamughi, and P. Antony Seba, "Enhanced shrimp disease classification through YOLO and data augmentation," in *Proc. 5th Int. Conf. Innov. Trends Inf. Technol. (ICITIIT)*, Mar. 2024, pp. 1–6.
[11] G.-J. Niu, M. Yan, C. Li, P.-Y. Lu, Z. Yu, and J.-X. Wang, "Infection with white spot syndrome virus affects the microbiota in the stomachs and intestines of kuruma shrimp," *Sci. Total Environ.*, vol. 839, Sep. 2022, Art. no. 156233.
[12] J. Suto, "Using data augmentation to improve the generalization capability of an object detector on remote-sensed insect trap images," *Sensors*, vol. 24, no. 14, p. 4502, Jul. 2024.
[13] C. Gao, Q. Zhang, Z. Tan, G. Zhao, S. Gao, E. Kim, and T. Shen, "Applying optimized YOLOv8 for heritage conservation: Enhanced object detection in jiangnan traditional private gardens," *Heritage Sci.*, vol. 12, no. 31, Jan. 2024, Art. no. 31, doi: 10.1186/s40494-024-01144-1.
[14] G. S. Ferrante, L. H. V. Nakamura, S. Sampaio, G. P. R. Filho, and R. I. Meneguette, "Evaluating YOLO architectures for detecting road killed endangered Brazilian animals," *Sci. Rep.*, vol. 14, no. 1, Jan. 2024, Art. no. 1353, doi: 10.1038/s41598-024-52054-y.
[15] G. S. Ferrante, L. H. V. Nakamura, F. R. H. Andrade, G. P. R. Filho, R. E. De Grande, and R. I. Meneguette, "Brazilian Road's Animals (BRA): An image dataset of most commonly run over animals," in *Proc. 35th SIBGRAPI Conf. Graph., Patterns Images (SIBGRAPI)*, Natal, Brazil, Oct. 2022, pp. 246–251, doi: 10.1109/SIBGRAPI55357.2022.9991774.
[16] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
[17] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Scaled-YOLOv4: Scaling cross stage partial network," 2020, *arXiv:2011.08036*.
[18] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO series in 2021," 2021, *arXiv:2107.08430*.
[19] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," 2022, *arXiv:2207.02696*.
[20] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "You only learn one representation: Unified network for multiple tasks," 2021, *arXiv:2105.04206*.
[21] R. Ozdemir and M. Koc, "On the enhancement of semi-supervised deep learning-based railway defect detection using pseudo-labels," *Expert Syst. Appl.*, vol. 251, Oct. 2024, Art. no. 124105. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417424009710
[22] *TCDD Dataset: Turkish Railway Statistics and Information*. Accessed: Feb. 10, 2026. [Online]. Available: https://tcdd.gov.tr
[23] A. R. Khouas, M. R. Bouadjenek, H. Hacid, and S. Aryal, "Training machine learning models at the edge: A survey," 2024, *arXiv:2403.02619*.
[24] S. Zhu, T. Voigt, F. Rahimian, and J. Ko, "On-device training: A first overview on existing systems," *ACM Trans. Sensor Netw.*, vol. 20, no. 6, Nov. 2024, Art. no. 118, doi: 10.1145/3696003.
[25] L. Ravaglia, M. Rusci, D. Nadalini, A. Capotondi, F. Conti, and L. Benini, "A tinyml platform for on-device continual learning with quantized latent replays," 2021, *arXiv:2110.10486*.
[26] J. P. H. Asdikian, M. Li, and G. Maier, "Performance evaluation of YOLOv8 and YOLOv9 on custom dataset with color space augmentation for real-time wildlife detection at the edge," in *Proc. IEEE 10th Int. Conf. Netw. Softwarization (NetSoft)*, Saint Louis, MO, USA, Jun. 2024, pp. 55–60.
[27] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," 2024, *arXiv:2405.14458*.
[28] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto, and E. A. B. da Silva, "A comparative analysis of object detection metrics with a companion open-source toolkit," *Electronics*, vol. 10, no. 3, p. 279, Jan. 2021.
[29] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Randaugment: Practical automated data augmentation with a reduced search space," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2020, pp. 3008–3017.
[30] JeonjuUniv. (Dec. 2022). *Deer Dataset*. [Online]. Available: https://universe.roboflow.com/jeonjuuniv/deer-hqp4i
[31] Trackabox. (Jun. 2022). *Wild Boar Dataset*. [Online]. Available: https://universe.roboflow.com/trackabox-4ejy9/wild-boar-a1flm
[32] S. Banerjee. (2022). *Animal Image Dataset (90 Different Animals)*. [Online]. Available: https://images.cv/data-sources/animal-image-dataset
[33] M. Xu, Z. Zhang, H. Hu, J. Wang, L. Wang, F. Wei, X. Bai, and Z. Liu, "End-to-end semi-supervised object detection with soft teacher," 2021, *arXiv:2106.09018*.
[34] J. Deng, W. Li, Y. Chen, and L. Duan, "Unbiased mean teacher for cross-domain object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 4089–4099.
[35] H. Zhou, Z. Ge, S. Liu, W. Mao, Z. Li, H. Yu, and J. Sun, "Dense teacher: Dense pseudo-labels for semi-supervised object detection," in *Proc. Eur. Conf. Comput. Vis.*, Cham, Switzerland, 2022, pp. 35–50.
[36] J. Wei, X. Zhang, Z. Ji, Z. Wei, and J. Li, "DPLRS: Distributed population learning rate schedule," *Future Gener. Comput. Syst.*, vol. 132, pp. 40–50, Jul. 2022.
[37] S. Adablanu, U. Barman, and D. Das, "15 years of optimizers in medical deep learning: A systematic review," *Neurosci. Informat.*, vol. 6, no. 1, Mar. 2026, Art. no. 100249.
[38] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–18. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7
[39] X. Li, C. Lv, W. Wang, G. Li, L. Yang, and J. Yang, "Generalized focal loss: Towards efficient representation learning for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3139–3153, Mar. 2023.
[40] X. Li, W. Wang, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized focal loss V2: Learning reliable localization quality estimation for dense object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 11627–11636.
[41] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2999–3007.
[42] N. Corporation. (2025). *JetPack 6.2 | NVIDIA Jetson Linux Developer Guide*. [Online]. Available: https://docs.nvidia.com



JEAN PIERRE ASDIKIAN (Graduate Student Member, IEEE) received the B.S. and M.S. degrees in computer and communications engineering from American University of Science and Technology, Achrafieh, Lebanon, in 2015 and 2017, respectively. He is currently pursuing the Ph.D. degree with the Department of Electronics, Information and Bioengineering, Politecnico di Milano, focusing on applied artificial intelligence models on far-edge devices. He has authored publications in object detection, service function chains, and network slicing. His research interests include computer vision, object detection in different domains, and far-edge training.



SEBASTIAN TROIA (Member, IEEE) received the Ph.D. degree (cum laude) in information technology (telecommunications) from the Politecnico di Milano, Italy, in 2020. He is a tenure-track Assistant Professor with the Department of Electronics, Information and Bioengineering, Politecnico di Milano; and a Fulbright Fellow with The University of Texas at Dallas, USA. He is a partner at SWAN Networks, a Politecnico di Milano spin-off, specializing in SDN/SD-WAN orchestration. He has contributed to several European projects, including H2020 Metro-Haul and NGI Atlantic. He has authored over 70 publications. His research focuses on edge network softwarization and machine learning for SDN, SD-WAN, and multi-layer optical/IP networks. In 2025, he received the IEEE ComSoc EMEA Outstanding Young Researcher Award. He is active in TPC and organizing committees for major conferences, such as IEEE ICC, GLOBECOM, NetSoft, and others. He is the Founder and a Co-Organizer of the Edge Network Softwarization (ENS) Workshop series, currently in fourth edition in conjunction with IEEE NetSoft 2025. He served as an Editor for ITU-T FG-ML5G.



OMRAN AYOUB (Member, IEEE) received the M.Sc. degree in telecommunications engineering and the Ph.D. degree in information technology (telecommunications) from the Politecnico di Milano, in 2015 and 2019, respectively. He is the Head of the trustworthy and secure information networks research area with the Institute of Information Systems and Networking (ISIN), SUPSI, where he is also a Senior Lecturer-Researcher. He has authored more than 80 scientific publications in international journals and conferences in the areas of optical networks and network optimization. His research interests include the design of trustworthy and explainable AI/ML systems for network automation, the application of AI/ML to network management, and optimization of multi-layer communication networks. He is the Founder and the General Chair of the International Workshop on Trustworthy and Explainable Artificial Intelligence for Networks and has delivered several invited talks and tutorials at leading international conferences on explainable AI for network management.



KIVILCIM YILMAZ received the B.Sc. degree in electrical and electronics engineering from Ankara University. She is currently a Systems Engineer with the Air Defense Systems Group, ASELSAN, Türkiye. As part of her studies, she completed an internship with the Politecnico di Milano (BONSAI Laboratory), where she worked under the supervision of Prof. Troia's team and gained exposure to research practices in telecommunications engineering and experimental network systems. She has academic experience in radar simulation using MATLAB's Phased Array Toolbox and in RF/microwave engineering, including the design of an X-band circularly polarized microstrip antenna in CST Studio Suite. Her professional interests include radar systems, microwave engineering, and communication technologies. She plans to further deepen her expertise through graduate studies and interdisciplinary engineering works.



GIACOMO SGUOTTI (Graduate Student Member, IEEE) received the M.Sc. degree (cum laude) in telecommunications engineering from the Politecnico di Milano, Milan, Italy, where he is currently pursuing the Ph.D. degree in information technology with the Department of Electronics, Information and Bioengineering. He has co-authored publications on SD-WAN architectures and satellite networking. His research interests include software-defined wide area networks, satellite and terrestrial network integration, and intelligent network management and optimization.



MENGYAO LI (Graduate Student Member, IEEE) received the dual B.Sc. degrees in electronics from Tongji University, China, and the Politecnico di Milano, Italy, and the M.Sc. degree in computer science and engineering from the Politecnico di Milano, where she is currently pursuing the Ph.D. degree in telecommunications. Her research interests include optimization in 5G radio access networks, network optimization, network slicing, machine learning algorithms for communications, and quantum key distribution networks.



MARCO MARCON (Member, IEEE) was born in Bollate, Milan, in 1972. He received the degree in electronic engineering from the Politecnico di Milano, Italy, in 1998, and the Ph.D. degree in applied physics from the Department of Physics, Politecnico di Milano, in 2003. In 2003, he joined the Department of Electronics, Information and Bioengineering (DEIB), Politecnico di Milano, where he is currently a Research Scientist with the Image and Sound Processing Laboratory (ISPL). He participated to the Origami European Project and the Visnet Network of Excellence. He is participating to its continuation: Visnet II NoE. His current research interests include digital image processing and computer vision for biometry and 3-D object reconstruct from multiple uncalibrated views.



GUIDO MAIER (Senior Member, IEEE) received the Laurea degree in electronic engineering and the Ph.D. degree in telecommunication engineering from the Politecnico di Milano, Italy, in 1995 and 2000, respectively. Until February 2006, he was a Researcher with CoreCom (a research consortium supported by Pirelli in Milan, Italy), where he achieved the position of the Head of the Optical Networking Laboratory. In March 2006, he joined the Politecnico di Milano, as an Assistant Professor and became an Associate Professor, in 2015. He is currently involved in industrial and European research projects. In 2016, he co-founded the startup SWANnetworks, a spin-off of the Politecnico di Milano. He is the author of more than 150 papers in the area of networking published in international journals and conference proceedings (an H-index is 30) and holds six patents. His research interests include optical networks modeling, design, and optimization; SDN orchestration and control-plane architectures; SD-WAN; and NFV. He is a Senior Member of the IEEE Communications Society and a TPC member of many international conferences. He is the General Chair of DRCN2020, DRCN2021, and NetSoft 2022. He is an Editor of *Optical Switching and Routing* and a Guest Editor for a special issue of IEEE Open Journal of the Communications Society.